

When answering a question, please provide an example. 3

1. a) "Comments are often used as deodorant" -- justify this statement. What are the remedies of

19 Give example

1. (a)

The statement "**Comments are often used as deodorant**" suggests that developers sometimes use comments to cover up bad code instead of writing clean, self-explanatory code. Just like deodorant masks bad odor without addressing the root cause, excessive or unnecessary comments may try to explain messy, unclear, or poorly structured code instead of improving it.

Justification:

1. **Bad Code Justification:** Instead of refactoring, developers write lengthy comments to explain convoluted logic.
2. **Code Smell Indicator:** If too many comments are required, it often signals poor naming, lack of modularization, or unclear logic.
3. **Maintenance Issues:** Over time, comments may become outdated or misleading if the code changes but the comments do not.

Remedies for Overusing Comments:

1. **Write Self-Explanatory Code:**
 - Use meaningful variable, function, and class names.
 - Keep functions short and focused on a single task.
2. **Refactor Instead of Explaining:**
 - Break complex logic into smaller, reusable functions or classes.
 - Apply **design patterns** (e.g., Strategy Pattern for conditional-heavy logic).
3. **Use Comments Only When Necessary:**
 - Document the **why**, not the **what** (e.g., explain non-obvious design decisions).
 - Use comments for legal disclaimers, warnings, or complex algorithms.

Example:

Bad Code with "Deodorant" Comments:

```
// This function calculates the total price after applying discount and tax.
// It first calculates the discount, then adds tax, and finally rounds the value.
double calc(double p, double d, double t) {
    double a = p - (p * d / 100); // Applying discount
    double b = a + (a * t / 100); // Adding tax
    return round(b); // Rounding final price
}
```

Refactored Code Without Unnecessary Comments:

```
double applyDiscount(double price, double discount) {
    return price - (price * discount / 100);
}

double applyTax(double price, double tax) {
    return price + (price * tax / 100);
}

double calculateFinalPrice(double price, double discount, double tax) {
    return round(applyTax(applyDiscount(price, discount), tax));
}
```

- Suppose we are creating a very simple file system. We have to maintain the data of files and folders. Files are the basic elements that can be saved in the file system. A folder contains several files in it. A folder may also contain one more folder in it. A file has the following information: *file name, file_extension, creation_time, file size, location*. A folder also has the following attributes: *creation_time, location, folder name*. A folder must maintain the list of folders and files in it. A folder should have the functionality to add and remove new folder and new file. A folder should be able to provide the list of files with file attributes in it.

Design the system mentioned above. Write down the name of the design pattern(s), draw the class diagram and write the code to support your design.

2.

```
FileSystemItem.java x File.java Folder.java FileSystemComposite.java
1
2 public abstract class FileSystemItem { 6 usages 2 inheritors
3     protected String name; 3 usages
4     protected String location; 3 usages
5     protected long creationTime; 1 usage
6
7     public FileSystemItem(String name, String location) { 2 usages
8         this.name = name;
9         this.location = location;
10        this.creationTime = System.currentTimeMillis(); // Current timestamp as creation time
11    }
12
13    public abstract void showDetails(); 2 usages 2 implementations
14 }
15
```

```
FileSystemItem.java File.java x Folder.java FileSystemComposite.java
1 public class File extends FileSystemItem{ no usages
2     private String fileExtension; 2 usages
3     private double fileSize; 2 usages
4
5     public File(String name, String location, String fileExtension, double fileSize) { no usages
6         super(name, location);
7         this.fileExtension = fileExtension;
8         this.fileSize = fileSize;
9     }
10
11     @Override no usages
12     public void showDetails() {
13         System.out.println("File: " + name + "." + fileExtension + " | Size: " + fileSize + "KB | Location: " + location);
14     }
15 }
```

```
FileSystemItem.java  File.java  Folder.java  FileSystemComposite.java x
1  public class FileSystemComposite {
2      public static void main(String[] args) {
3          Folder root = new Folder( name: "Root", location: "C:/");
4
5          File file1 = new File( name: "document", location: "C:/", fileExtension: ".txt", fileSize: 15.5);
6          File file2 = new File( name: "report", location: "C:/", fileExtension: ".pdf", fileSize: 30.2);
7
8          Folder subFolder = new Folder( name: "Projects", location: "C:/Projects");
9          File projectFile = new File( name: "project", location: "C:/Projects", fileExtension: ".java", fileSize: 12.7);
10
11          // Constructing hierarchy
12          subFolder.addItem(projectFile);
13          root.addItem(file1);
14          root.addItem(file2);
15          root.addItem(subFolder);
16
17          // Displaying file system structure
18          root.showDetails();
19      }
20  }
```

3. a) What is Primitive Obsession? Give one example of primitive obsession and refactor it. What are the remedies of primitive obsession? 3

3. (a)

This smell exists when primitives, such as strings, doubles, arrays or low-level language components, are used for high-level operations instead of using classes.

Example of Primitive Obsession:

```
java Copy
public class Order {
    private String customerName;
    private String customerAddress;
    private String customerCity;
    private String customerState;
    private String customerZipCode;
    // Other order-related fields and methods
}
```

In this example, customer information is represented using multiple primitive strings, which can lead to duplication and lack of encapsulation.

Refactored Example:

java

Copy

```
public class Customer {  
    private String name;  
    private Address address;  
    // Other customer-related fields and methods  
}  
  
public class Address {  
    private String street;  
    private String city;  
    private String state;  
    private String zipCode;  
    // Address-related methods  
}  
  
public class Order {  
    private Customer customer;  
    // Other order-related fields and methods  
}
```

☒ Remedies

- ☒ Extract Class
- ☒ Replace Data Value with Object
- ☒ Replace Type Code with Class
- ☒ Introduce Parameter Object
- ☒ Replace Array with Object

3(b) Mediator Pattern (Out of Syllabus)

- | | | |
|-------|---|---|
| 4. a) | Which design pattern provides a way for object cloning? Explain a scenario where this pattern can be used. | 3 |
| b) | "Talk to the interface, not to the concrete class" -- Explain this statement with appropriate reasoning. | 3 |
| c) | What are the patterns that seem to be similar to Proxy pattern? Explain why they are similar. What are the dissimilarities that made these patterns exist separately? | 4 |

4.

(a)

Design Pattern for Object Cloning: The Prototype Pattern provides a way to clone objects. This pattern involves creating new objects by copying an existing object, known as the prototype.

Scenario: Imagine you are developing a graphic editor application where users can create and manipulate shapes. Each shape (e.g., circle, rectangle) can have complex properties and behaviors. Instead of creating new shapes from scratch each time, you can use the Prototype Pattern to clone existing shapes.

(b)

Explanation:

This statement emphasizes the importance of programming to an interface rather than a concrete implementation. It is a fundamental principle in object-oriented design that promotes flexibility, maintainability, and scalability.

Reasoning:

- **Decoupling:** By depending on interfaces, you decouple your code from specific implementations. This makes it easier to change or extend the system without affecting other parts.
- **Flexibility:** Interfaces allow you to switch implementations at runtime, providing greater flexibility. For example, you can replace a database access implementation with a mock object for testing.
- **Maintainability:** Code that depends on interfaces is easier to maintain because it is less likely to break when implementations change.
- **Reusability:** Interfaces define contracts that can be implemented by multiple classes, promoting code reuse.

5. a) Which design pattern/patterns is/are implemented in the following code? Explain your answer. 5

```
public abstract class ReportTemplate {
    protected ReportFormat reportFormat;

    public void setReportFormat(ReportFormat reportFormat) {
        this.reportFormat = reportFormat;
    }

    public void generateReport() {
        generateHeader();
        reportFormat.generateBody();
        generateFooter();
    }
}
```

CS CamS

```
protected abstract void generateHeader();
protected abstract void generateFooter();
}

public interface ReportFormat {
    void generateBody();
}
```

... patterns with code/ UML examples. 5

The code provided implements the Template Method Pattern and uses the Strategy Pattern in combination.

Template Method Pattern: The ReportTemplate class defines a template method generateReport() that outlines the structure of an algorithm. The steps of the algorithm are defined in methods like generateHeader(), generateBody(), and generateFooter(). The generateHeader() and generateFooter() methods are abstract, meaning they must be implemented by subclasses, while generateBody() is delegated to the ReportFormat interface.

Strategy Pattern: The ReportFormat interface defines a family of algorithms (in this case, the generation of the report body). Different implementations of ReportFormat can be used interchangeably to change the behavior of the generateBody() method without altering the ReportTemplate class.

```
public class PDFReportFormat implements ReportFormat {
    @Override
    public void generateBody() {
        System.out.println("Generating PDF report body...");
    }
}

public class HTMLReportFormat implements ReportFormat {
    @Override
    public void generateBody() {
        System.out.println("Generating HTML report body...");
    }
}

public class SalesReport extends ReportTemplate {
    @Override
    protected void generateHeader() {
        System.out.println("Sales Report Header");
    }

    @Override
    protected void generateFooter() {
        System.out.println("Sales Report Footer");
    }
}

public class ReportDemo {
    public static void main(String[] args) {
        SalesReport salesReport = new SalesReport();
        salesReport.setReportFormat(new PDFReportFormat());
        salesReport.generateReport();

        salesReport.setReportFormat(new HTMLReportFormat());
        salesReport.generateReport();
    }
}
```


6.

6. a) What is a Lazy Class? Give an example of a lazy class. How can a lazy class be refactored?

A **Lazy Class** is a code smell where a class does not do enough to justify its existence. It may have too few responsibilities or its functionality could easily be merged into another class without losing clarity or increasing complexity. Lazy classes can lead to unnecessary complexity and maintenance overhead.

Example of a Lazy Class:

java

Copy

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

In this example, the **Calculator** class only has a single method, **add**, which might not be enough to justify having a separate class. This functionality could easily be moved to a more general utility class.

Refactoring a Lazy Class:

- **Inline Class:** Move the functionality of the lazy class into another class and then delete the lazy class.
- **Merge Classes:** If there are multiple lazy classes, consider merging them into a single class that handles related responsibilities.

b) What kind of code smells do you get from the below code segment? Refactor the code and mention the approach of refactoring you applied.

Code Smells:

1. **Duplicated Code:** The methods **GetCsv**, **GetCsvWithHeader**, and **GetCsvWithHeaderAndFooter** have similar code for generating CSV rows.
2. **Long Method:** The methods are longer than necessary due to repeated code.
3. **Primitive Obsession:** The code directly manipulates strings instead of using a more structured approach.

```

public class Person {
    private String name;
    private String email;
    private int age;

    // Constructor, getters, and setters
}

public class CsvGenerator {
    public String getCsv(List<Person> persons, boolean includeHeader, boolean includeFooter) {
        StringBuilder csv = new StringBuilder();

        if (includeHeader) {
            csv.append("Name, Email, Age\n");
        }

        for (Person person : persons) {
            csv.append(person.getName()).append(",")
                .append(person.getEmail()).append(",")
                .append(person.getAge()).append("\n");
        }

        if (includeFooter) {
            csv.append("Total no of persons:").append(persons.size());
        }

        return csv.toString();
    }
}

```

Refactoring Approach:

- **Extract Method:** Created a single method `getCsv` that handles all variations by using parameters to control the inclusion of headers and footers.
- **Use StringBuilder:** Replaced string concatenation with `StringBuilder` for better performance.
- **Encapsulation:** Used private fields with getters and setters in the `Person` class to encapsulate the data.

This refactoring reduces code duplication, improves readability, and makes the code more maintainable.